Store POS: Product Requirements Document

Oct 13, 2026 · @Shin Thant

An offline desktop POS for our family farm supply store that lets two people record sales, track customer debts and see daily and weekly results, with no internet needed.

Problem and goals

Sales are written in a ledger book today. Entries are sometimes forgotten, and daily or weekly totals and best sellers are hard to work out by hand. This app replaces the ledger.

- Record a sale in under a minute, with the total calculated automatically.
- Record every unpaid balance against a named customer so no debt is lost.
- Show daily and weekly sales, earnings and best sellers without any manual adding.
- Run fully offline on one laptop.

Success after one month: no sale lives only on paper, and both users can read the weekly report without help.

Users and constraints

The only users are my father and mother, who run the counter, the stock and the books themselves. There are no employees, so the app has no login, roles or permissions.

- **Hardware:** one laptop. No receipt printer and no barcode scanner, so every item is found by name or category.
- **Internet:** not guaranteed, so nothing may depend on it. All data lives in a local SQLite file.
- **Platform:** a Windows desktop application built with C# .NET and WPF, using SQLite for storage.
- **Skills:** non-technical users, so screens must be simple and use large, clear text.
- **Product data:** product information comes on paper lists and images. I will type it into a CSV file, which the app imports.

Scope

Version 1 covers the four features the family asked for and nothing more.

In scope

- Transaction engine (build a sale, price it, save it)

- Price list with search and filters, plus CSV import
- Customer debt records with repayments
- Daily and weekly sales reports with best and worst sellers
- Local backup and restore of the database

Out of scope for version 1

- Logins, user roles or permissions
- Cloud sync or any feature that needs internet
- Barcode scanning and receipt printing
- Stock tracking (counting what is in the warehouse), supplier purchasing, and due dates or reminders on debts

Feature 1: Transaction engine

A sale is a cart of items that is priced automatically and saved with the time it happened. This is the screen used most, so it must be the fastest.

1. **TX1:** Search or pick a product and add it to the cart with a quantity. Quantities are whole numbers. Products marked as sold by weight, such as some fertilizers, also accept decimals (for example 2.5 kg).
2. **TX2:** Each line shows unit price × quantity = line total, rounded to the nearest whole kyat. The cart shows the sale total, updated live.
3. **TX3:** A line's price can be changed for that sale only (for example a negotiated price). The product's list price does not change.
4. **TX4:** Enter the amount paid. The app shows the change to give back, or the unpaid balance if less than the total was paid.
5. **TX5:** If a balance remains, the app requires a customer and creates a debt record (Feature 3) in the same step.
6. **TX6:** Saving stores the date, time, items, prices, total and amount paid. Prices are copied into the sale, so later price changes never alter past sales.
7. **TX7:** A saved sale cannot be edited. A mistake is fixed by voiding the sale with a short reason and entering a new one. Voided sales stay visible but are left out of reports.

Done when: a sale with three items and a part payment can be saved in under a minute and appears in the day's report.

Feature 2: Price display

One screen lists every product with its current price, so either parent can answer a customer's question in seconds. The same product list feeds the sale screen.

1. **PR1:** Show all products in a table with name, category, unit or pack size, and price.
2. **PR2:** A search box matches any part of the name as the user types, ignoring capital letters.
3. **PR3:** Filter by category (pesticide, herbicide, fertilizer, seed, other). Filters and search can be combined.
4. **PR4:** Sort by name, category or price.
5. **PR5:** Add, edit and deactivate a product. A deactivated product disappears from the sale screen but stays in past sales and reports.
6. **PR6:** Import products from a CSV file that I prepare from the paper price lists and images. The file has the columns name, category, unit, price and sold by weight (yes or no), and is saved as UTF-8 so Burmese names import correctly. The app shows a preview, then adds new products and updates the price of existing ones. Rows it cannot read are listed, not skipped silently.
7. **PR7:** Product names may be in Burmese or English. Both are stored and searched correctly.

Done when: the full CSV price list is imported and a product can be found by typing part of its name.

Feature 3: Debt records

When a customer pays only part of a sale, the app keeps a record of who owes what, what they bought, and every repayment until the balance reaches zero.

1. **DB1:** Each debtor has a customer record with name and address. Phone number is optional. An existing customer is picked from a list instead of being typed again.
2. **DB2:** A debt is created automatically from a sale with an unpaid balance (TX5). It links to that sale, so the items bought and their prices are always visible.
3. **DB3:** Record a repayment against a customer with the date and amount. The balance drops by that amount, and one repayment can be spread across several of that customer's open debts, oldest first.
4. **DB4:** The debt list shows each customer's name, address and total still owed. A summary at the top shows the total owed to the store.
5. **DB5:** Search by name or address. Filter between customers who still owe and customers who have paid in full.
6. **DB6:** A customer's page shows every sale on credit and every repayment, newest first, and the running balance.
7. **DB7:** Repayments are never deleted. A wrong repayment is reversed with a reason, like a voided sale.

Done when: a part-paid sale creates a debt, two later repayments clear it, and the customer's history shows all three events.

Feature 4: Daily and weekly reports

Every saved sale and repayment is logged automatically, and the report screen turns that log into totals and rankings with no manual adding.

1. **RP1:** A daily report for any chosen date and a weekly report for any chosen week. A week runs Monday to Sunday.
2. **RP2:** Each report shows the metrics in the table below.
3. **RP3:** Best sellers: products ranked by quantity sold and by sales amount, top 10.
4. **RP4:** Slow sellers: products with the lowest sales in the period, including products with no sales at all.
5. **RP5:** A bar chart of sales per day for the weekly report, and a list of the day's sales that opens each sale's items.
6. **RP6:** Voided sales are excluded from every figure.
7. **RP7:** A report can be exported to Excel or PDF so it can be kept or shown to others.

Metric	Meaning
Total sold	Sum of all sale totals in the period, whether or not they were paid. This is what the family calls "earned".
Cash received	Payments taken at sale time plus debt repayments received in the period
Credit given	Unpaid balances created by sales in the period
Number of sales	Count of saved, non-voided sales

Done when: the weekly report's total matches the sum of the seven daily reports.

Data model

Seven SQLite tables hold everything. Reports are calculated from the sales and repayment tables, so there is no separate report data to keep in sync.

Table	Key fields	Purpose
products	name, category, unit, price, sold by weight, active	The price list
customers	name, address, phone (optional)	People who buy on credit
sales	date and time, total, amount paid, customer (optional), status, void reason	One row per sale
sale_items	sale, product, quantity, unit price, line total	Items in each sale, with the price at that moment
debts	customer, sale, original amount	One row per sale with an unpaid balance
repayments	customer, amount, date, reversed flag and reason	Money received against debts
repayment_allocations	repayment, debt, amount	Which debts each repayment cleared

Money is stored as whole kyat amounts to avoid rounding errors. Quantities are stored with decimals so a weight such as 2.5 kg is kept exactly. A debt's remaining balance is its original amount minus its allocations, never a typed-in number.

Non-functional requirements

- **Offline:** every feature works with no internet, and the app never tries to connect.
- **Data safety:** a sale is saved completely or not at all, so a power cut mid-sale cannot leave half a record. This uses SQLite transactions.
- **Backups:** the app copies the database to a chosen folder or USB drive once a day and on request, and can restore from any copy. One laptop failure must not erase the shop's records.
- **Speed:** search results appear within half a second with at least 5,000 products. Starting the app takes under 5 seconds.
- **Ease of use:** large text, few buttons per screen, and a confirmation before any void or reversal. No training should be needed beyond a short walkthrough.
- **Language:** the interface is in Burmese. If a screen cannot be shown properly in Burmese in version 1, that screen uses English. Burmese text must display and search correctly on the laptop. All screen text is kept in one place so it can be translated or switched later.

- **Install:** one installer file, with no database server or other software to set up.

Milestones

Build in this order, so the shop can start using the app before every feature is finished.

1. **Products:** database, product list, search, filters and CSV import.
2. **Sales:** the transaction engine with paying in full.
3. **Debts:** part payments, customer records and repayments.
4. **Reports:** daily and weekly reports with best and slow sellers.
5. **Safety and trial:** backup and restore, then run the app next to the ledger book for two weeks and compare totals before dropping the ledger.